

For example, an AI-enabled system can:

- Detect a pedestrian
- Calculate risk based on distance and speed
- Automatically trigger braking if required

This is the stage at which AI moves from prediction to autonomous action. Not every automotive feature requires AI. Developers and organisations must carefully determine where AI genuinely adds value rather than introducing unnecessary complexity. The goal is not to use AI everywhere; it is to use it effectively in areas where prediction, automation, and rapid decision-making can improve safety, efficiency, and overall system performance.

From analysis to autonomous action

Data analytics explains what is happening inside a system. For example, it may identify that a certain distance from a pedestrian is risky. Machine learning goes one step further by predicting outcomes, such as whether applying brakes can reduce risk or improve safety. AI builds on this by taking action automatically. Once the system identifies a critical situation, it can trigger braking or corrective control.

This progression shows how embedded intelligence evolves from analysis to prediction and finally to autonomous action. At the same time, developers must carefully evaluate where AI is genuinely required. Not every feature needs AI simply because it is trending. The real challenge is identifying where AI can genuinely improve safety, efficiency, or decision-making within an automotive or embedded workflow.

Different automotive functions map to different levels of intelligence. Engine parameter monitoring, fuel consumption analysis, onboard diagnostics, battery ageing analysis, and DTC fault logging fall under data analytics because they

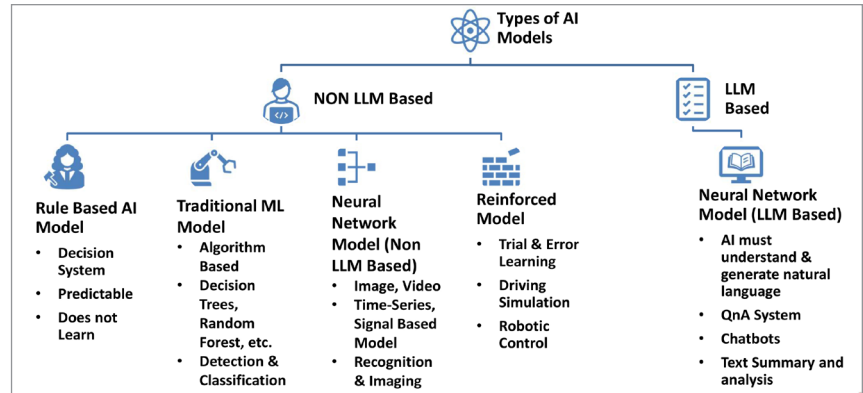


Fig. 2: Types of AI models used in embedded system development

focus on monitoring and interpretation. Tyre wear estimation, surface classification, and driver behaviour classification fall under machine learning because they involve prediction and classification without direct autonomous action. In contrast, autonomous emergency braking, lane-keep assist, adaptive cruise control, and driver monitoring systems fall under AI because they actively respond to environmental conditions and make decisions automatically.

Lane-keep assist is a clear example of this evolution. Basic systems only warn about lane departure, while AI-driven systems continuously analyse their surroundings and actively keep the vehicle within its lane. Similarly, adaptive cruise control has existed for years, but modern AI-driven implementations now analyse terrain, traffic, pedestrians, and driving patterns before determining vehicle behaviour. Instead of manually selecting modes such as Sport or Economy, AI can choose the most suitable mode based on real-time conditions.

AI itself is not a single technology. It includes rule-based AI, traditional ML models, neural networks, reinforcement learning, and large language models (LLMs). Rule-based AI operates on predefined logic; ML models rely on trained datasets; neural networks process image streams, video inputs, and continuous sensor data; reinforcement learning improves through

trial and error; and LLMs focus on language understanding and reasoning. Systems such as ChatGPT are LLMs, but deploying them directly on microcontrollers is impractical because of memory and compute constraints.

The real transformation begins when AI becomes part of the embedded development workflow. Traditionally, software updates involve manual impact analysis, architectural modifications, requirements updates, test-case generation, deployment, validation, and release management. With AI integration, many of these repetitive tasks can become semi-automated. AI systems can identify impacted modules, suggest architectural updates, generate test cases, automate release packaging, and assist in validation.

This also extends into coding and compliance. NLP-based AI systems can detect ambiguous or inconsistent requirements and suggest improvements from a functional safety or cybersecurity perspective. AI-assisted static analysis tools can verify MISRA compliance and recommend fixes. This reduces repetitive effort in debugging, compliance checks, documentation, and audit preparation.

Despite this automation, human intervention remains essential. AI outputs, especially from LLM-based systems, are probabilistic rather than fully deterministic. Every generated requirement, architectural change, code update, or release